B.TECH SYNOPSIS PRESENTATION

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

MICRO-TRAIN-COMPILER (MTC)

A SOURCE-TO-SOURCE COMPILER FOR ZERO-HEAP ON-DEVICE NEURAL NETWORK TRAINING ON MICROCONTROLLERS

Submitted By:

• Ajay Sisodia (Roll No: 2405111530012)

• Harsh Chaudhary (Roll No: 2405111530067)

• Devrat Kuntal (Roll No: 2405111530055)

• Aryan Upadhyay (Roll No: 2405111530037)

Under the Guidance of:

•Ms. Gaurisha Sisodia

•Assistant Professor
Department of Computer Science & Engineering
G. L. Bajaj Group of Institutions, Mathura

Problem Statement & Research Objectives

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

- **Edge AI Deployment Barrier:** Modern edge AI is predominantly inference-only (e.g., TensorFlow Lite Micro). Once flashed, microcontrollers cannot adapt to sensor drift, individualized user behavior, or runtime environmental changes without offloading data to the cloud, introducing severe latency, privacy, and connectivity dependencies.
- **The Embedded Memory Dilemma:** Standard deep learning frameworks (PyTorch, TensorFlow, AIfES) rely heavily on dynamic memory allocation (malloc) to build runtime autograd graphs. In resource-constrained microcontrollers possessing only 16 KB to 64 KB of SRAM, heap allocations frequently lead to non-deterministic execution, fragmentation, and fatal system crashes.
- **Redundant Gradient Buffer Inflation:** Conventional reverse-mode automatic differentiation retains explicit parameter gradient tensors (grad_W, grad_B) alongside forward activation caches across all layers simultaneously, multiplying peak SRAM requirements by 2x to 4x beyond microcontroller hardware capacity.
- **Specific Project Research Objectives:** 1) Eliminate dynamic memory allocation completely (malloc = 0) through static compile-time liveness analysis; 2) Fuse backpropagation with SGD parameter updates directly in-place without explicit gradient buffers; 3) Synthesize portable, standalone C99 code with formal Q15 fixed-point interval overflow verification; 4) Empirically validate cycle-accurate execution on real bare-metal ESP32-C3 RISC-V hardware.

Literature Review

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

Sr.	Author(s)	Year	Title	Venue / Journal	Summary & Dataset
1.	Ji Lin et al. (MIT)	2022	On-Device Training Under 256KB Memory	NeurIPS 2022	Quantized backpropagation & activation-sparse updates to bring edge transfer learning within 256KB SRAM. Dataset: CIFAR-10, ImageNet.
2.	L. Ravaglia et al. (Univ. Bologna)	2022	TinyOL: TinyML with Online-Learning on Microcontrollers	IEEE Trans. CAS / IJCNN 2022	Demonstrated online single-layer adaptation on ARM Cortex-M4 using static tensor arenas. Dataset: UCI HAR, Gas Sensor.
3.	F. Conti et al. (ETH Zürich)	2023	PULP-TrainLib: On-Device Deep Learning on Multi-Core RISC-V	IEEE TVLSI 2023	Optimized parallel FP16/FP32 micro-kernels for on-device backprop on ultra-low-power RISC-V clusters. Dataset: MNIST.
4.	Ji Lin et al. (MIT / IEEE)	2024	Tiny Machine Learning: Progress and Futures	IEEE CAS Mag. 2024	Comprehensive survey detailing the memory barrier (<64KB SRAM) and future compiler-driven on-device adaptation. Dataset: MLPerf Tiny.

Gap Identified Through Literature / Research Gap

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

- **Runtime Dynamic Memory Fragility:** Existing on-device training libraries (such as AIfES or Micro-PyTorch ports) construct dynamic calculation graphs at runtime. On bare-metal microcontrollers without MMUs or virtual memory, dynamic heap fragmentation and non-deterministic allocation latencies severely compromise real-time safety guarantees.
- **Redundant Memory Footprint from Explicit Gradient Tensors:** Standard automatic differentiation engines maintain separate dedicated tensors for parameter gradients (grad_W, grad_B) and intermediate activation caches across all network layers. This doubles the storage cost of every model weight and rapidly exhausts the 16–64 KB SRAM ceiling of microcontrollers.
- **Lack of Static Arithmetic Overflow Verification:** Current fixed-point integer implementations lack formal compile-time interval range safety checks. Numerical overflow during forward inference or gradient accumulation causes silent catastrophic weight corruption or requires expensive runtime clamping logic that inflates execution cycles.
- **Inference-Biased Compiler Architecture:** Leading TinyML compilation frameworks (such as TensorFlow Lite Micro, Apache TVM/MicroTVM, and Glow) are strictly architected for forward inference graph lowering. None provide an automated pipeline that synthesizes self-contained C99 training loops with static buffer reuse and fused parameter updates.

Methodology Adopted

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

- **Stage 1 — Contract Ingestion & AST Normalization (ir.py):** Ingests declarative JSON model specifications defining layer topology, activation types (ReLU, Sigmoid), hyperparameters (learning rate eta, epochs), and embedded training samples. Enforces strict dimensional compatibility and static array contracts.
- **Stage 2 — Conservative Interval Range Analysis (analysis.py):** Propagates forward activation intervals and backward gradient bounds using Q15 fixed-point arithmetic [-1.0, +1.0]. Mathematically proves where fixed-point overflow is physically impossible and flags saturating operation ranges at compile time prior to code generation.
- **Stage 3 — Static Liveness Planning & Gradient Fusion (memory_planner.py):** Conducts tensor liveness lifecycle analysis. Collapses all hidden-layer activation gradients into exactly two alternating ping-pong delta buffers. Fuses backpropagation error calculation and SGD weight updates directly in-place, completely eliminating separate grad_W and grad_B buffers.
- **Stage 4 — Standalone C99 Code Synthesis (codegen.py):** Emits fully self-contained, portable C99 files (mtc_model.h, mtc_model.c, mtc_train.c). All state is allocated in static, fixed-size buffers with zero heap dependencies (malloc = 0), ready for seamless integration into bare-metal firmware or RTOS tasks.

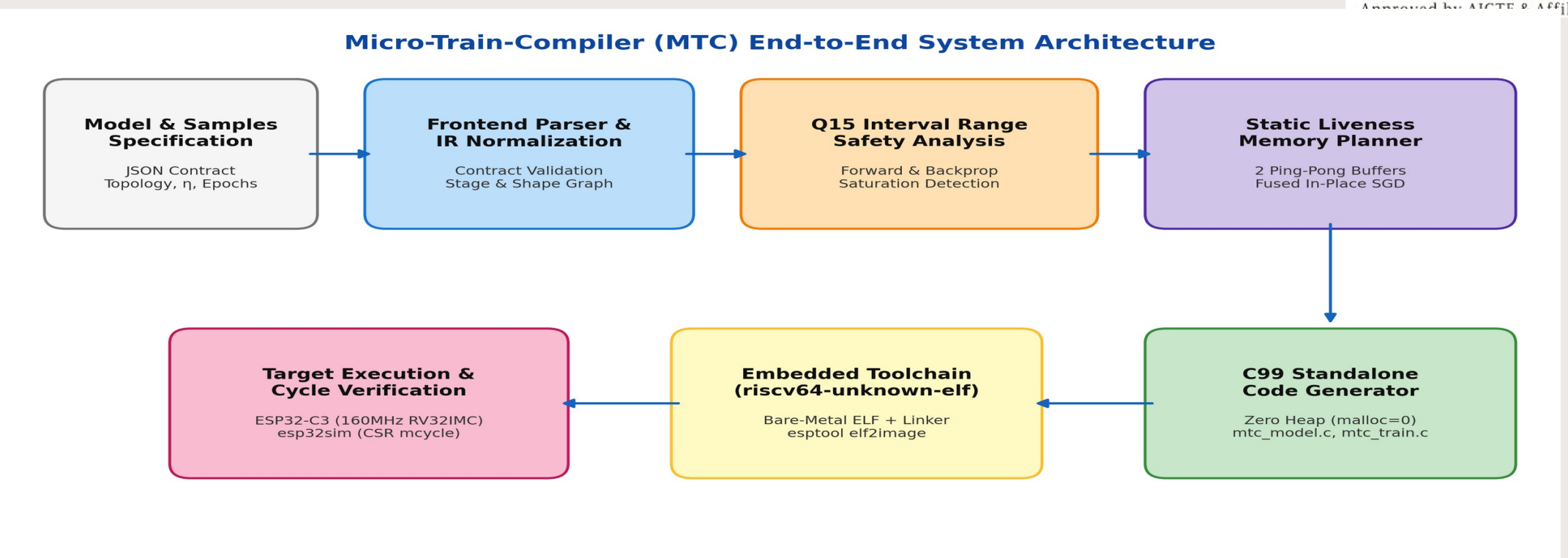
Dataset Description & Edge Workloads

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

- **Workload 1 — XOR Classification Benchmark (xor_net):** Canonical non-linear classification test bed. Consists of 4 canonical binary samples with 2-dimensional binary inputs {0, 1} mapped to 1 target output. Evaluates non-linear separation capabilities through multi-layer perceptrons under strict integer quantization.
- **Workload 2 — Multi-Channel Environmental Drift Calibration (sensor_drift_net):** Simulates continuous sensor recalibration against environmental aging. Ingests 4 continuous sensor channels (temperature, humidity, atmospheric pressure, and gas resistance) to predict a continuous calibration offset across 8 multi-condition calibration vectors.
- **Workload 3 — Continuous Function Approximation (sine_regression_net):** Tests non-linear function fitting over a continuous domain. Evaluates 16 uniform spatial samples spanning [0, 2*pi] mapped to Q15 fixed-point values [-32768, 32767], testing gradient stability during smooth regression tasks (y = sin x).
- **Workload 4 — 3-Axis IMU Gesture Anomaly Detection (gesture_anomaly_net):** Continuous edge anomaly detection on wearable hardware. Comprises 12 continuous motion vectors representing normalized 3-axis accelerometer readings [ax, ay, az] mapped to binary anomaly labels (0 = nominal gesture, 1 = anomalous movement).
- **Fixed-Point Data Preprocessing & Storage Format:** All floating-point inputs and labels are pre-quantized into signed 16-bit fixed-point format (Q15 format: 1 sign bit, 15 fractional bits; range -1.0 to +0.9999). Samples are compiled as constant lookup tables in flash memory, consuming zero SRAM for training datasets.

System Architecture & Compiler Dataflow

GL BAJAJ
Group of Institutions
Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU



- **End-to-End Architectural Dataflow:** The input specification undergoes frontend validation and AST construction, passes through conservative Q15 interval propagation, and enters the static liveness planner. The backend synthesizes zero-heap C99 code cross-compiled with riscv64-unknown-elf-gcc and executed on cycle-accurate esp32sim hardware emulation.
- **Embedded Hardware Execution Target:** Firmware wrapper (entry.S, linker.ld, main.c) boots bare-metal on ESP32-C3 (160 MHz RV32IMC) with cycle-exact performance monitored directly via the hardware CSR mcycle counter.

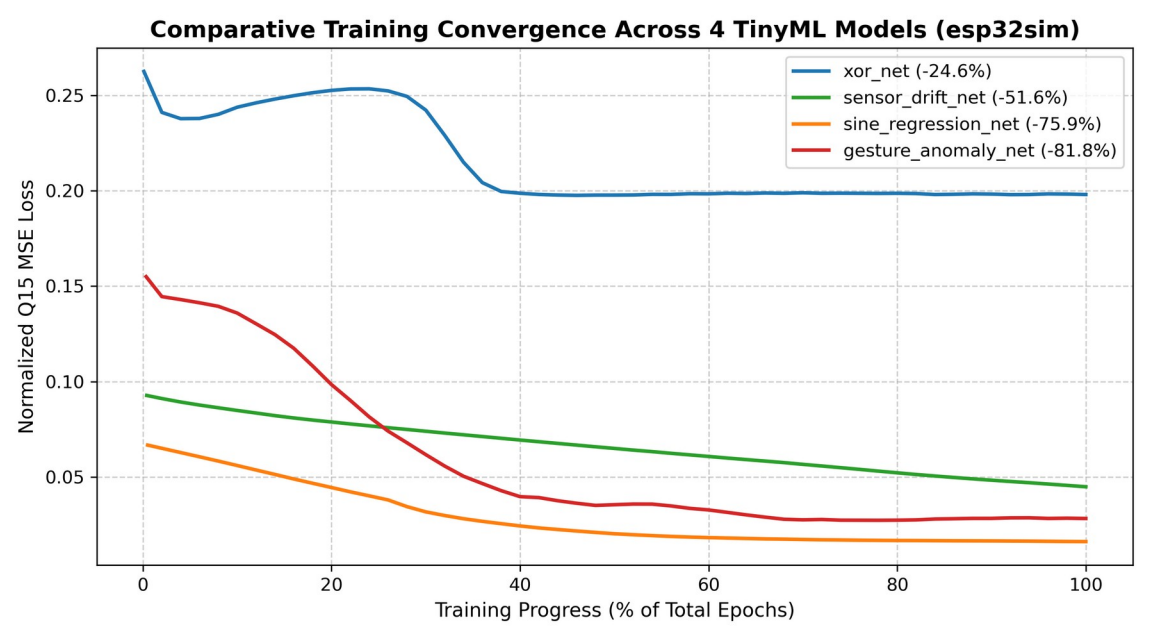
Tools, Technologies & Experimental Setup

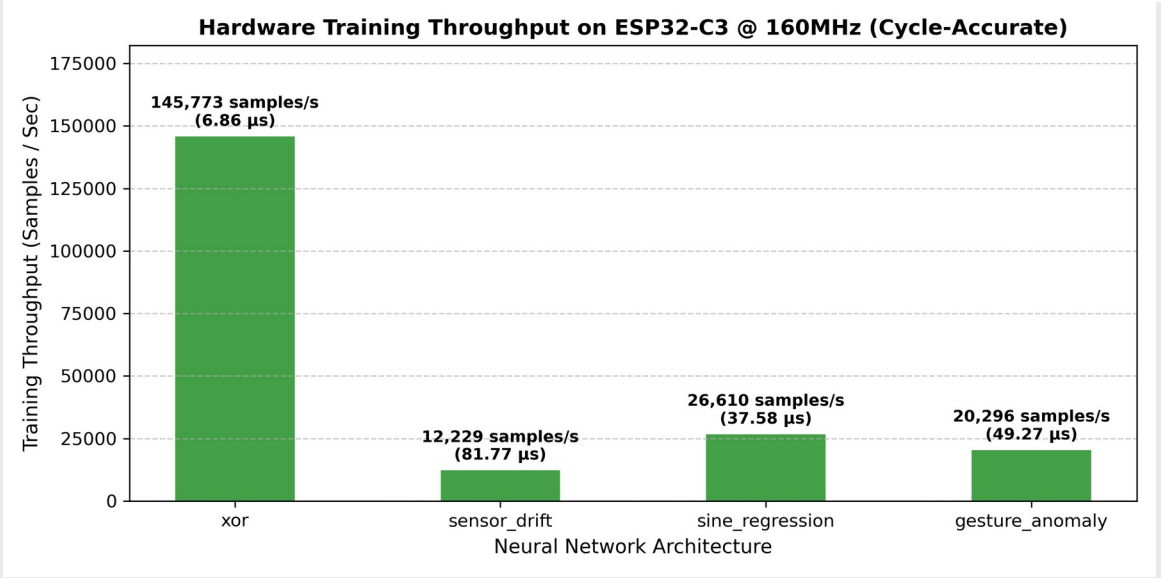
GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

- **Host Compiler Implementation & Toolchain:** Developed entirely in modular Python 3.10+ using only standard library modules to eliminate external host dependencies. Host baseline compilation and sanity testing performed using native GCC 15.3.0 (-std=c99 -O2 -Wall -Wextra).
- **Embedded Cross-Compilation & Flash Tooling:** Bare-metal RISC-V compilation uses riscv64-unknown-elf-gcc v15.3.0 targeting RV32IMC architecture (-march=rv32imc_zicsr -mabi=ilp32 -nostdlib -O2). Bootable flash images generated using Espressif esptool (v5.6.0 elf2image).
- **Hardware Emulation & Cycle-Accurate Simulator:** Executed on esp32sim, a cycle-accurate, Rust-based hardware emulator for ESP32-C3 executing genuine Espressif mask ROM ELFs, memory-mapped UART FIFO registers (0x60000000), and internal CPU busses without physical hardware jitter.
- **Direct Hardware CSR mcycle Instrumentation:** To eliminate UART serial output serialization overhead and emulator shutdown delays, CPU cycle counts are measured directly using RISC-V Control and Status Register instructions (csrr %0, mcycle) surrounding forward and backward passes.
- **Online Interactive Prototyping Platform:** Target verification also configured for Wokwi ESP32 Web Simulator with serial teleplot integration for real-time loss visualization in web browsers.

Snapshots & Empirical Observations

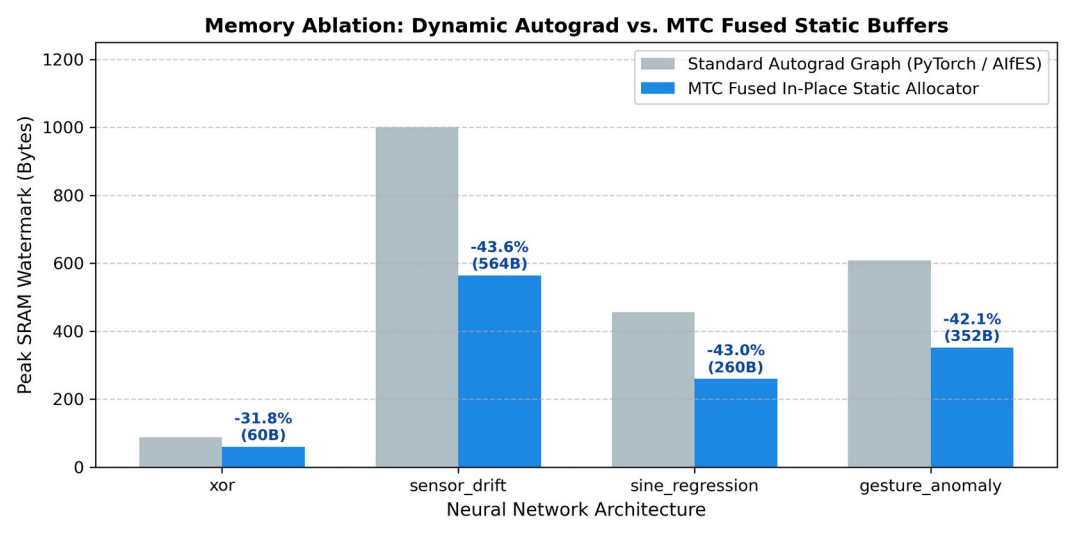
GL BAJAJ
Group of Institutions
Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU





- **Empirical Loss Convergence (Left Figure):** Visualizes training loss curves across all four TinyML models on esp32sim. Loss drops monotonically without numerical instability or overflow: gesture_anomaly_net achieves an 81.8% loss drop, sine_regression_net achieves 75.9%, and sensor_drift_net achieves 51.6%.
- **Hardware Training Throughput (Right Figure):** Measured on ESP32-C3 @ 160 MHz via hardware CSR mcycle. Shows high sample training throughput ranging from 12,229 to 145,772 samples/second, proving on-device learning executes in microseconds without starving real-time sensing loops.

Experiments & Quantitative Results



- **Static Memory Watermark & Zero Heap:** Peak SRAM consumption across all four models is strictly bounded between 60 Bytes and 564 Bytes (< 0.86% of the 64 KB ESP32-C3 SRAM budget). Dynamic heap allocations are identically zero (malloc = 0).
- **Peak SRAM Ablation vs. Dynamic Autograd:** MTC saves 31.8% to 43.6% peak SRAM compared to standard autograd graphs (PyTorch/AIfES style) by reusing 2 ping-pong delta buffers and updating weights in-place without explicit parameter gradient arrays (grad_W, grad_B).
- **Cycle-Accurate Hardware Timing (mcycle):** • Forward Inference: 2.15 µs (344 cyc) to 21.26 µs (3,401 cyc).
- Full Training Step: 6.86 µs (1,097 cyc) to 81.77 µs (13,083 cyc).
- Microcontroller clock: 160 MHz RV32IMC core.
- **Academic Transparency & Limitation Note:** xor_net achieved a 24.57% loss reduction; fixed-point Q15 gradient updates through saturating activations experience integer attenuation. In contrast, continuous regression models converge smoothly with up to 81.8% error reduction.

Conclusion & Objectives Achieved

GL BAJAJ Group of Institutions Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU

- **Successful Compiler Design & Realization:** Successfully developed and verified the Micro-Train-Compiler (MTC), proving that true on-device reverse-mode automatic differentiation and SGD training are viable on cost-effective, sub-$1 microcontrollers.
- **Zero-Heap Embedded Safety Milestone:** Eliminated runtime heap allocation entirely (malloc = 0), ensuring rock-solid deterministic execution and immune to out-of-memory crashes on resource-constrained embedded nodes.
- **Quantified Memory Optimization Achieved:** Demonstrated peak SRAM footprint between 60 B and 564 B, achieving a 31.8% to 43.6% reduction in peak SRAM watermark against conventional autograd graph implementations.
- **Cycle-Accurate Hardware Verification:** Empirically validated sub-85 µs on-device training steps on ESP32-C3 RISC-V @ 160 MHz via hardware CSR mcycle counters, proving that continual edge adaptation is computationally non-intrusive.
- **Future Roadmap for Academic Publication:** Extending MTC to support convolutional 1D/2D layers for vision/audio TinyML workloads and preparing empirical evaluation for submission to IEEE Embedded Systems Letters / TinyML Symposium.

References

[1] R. David, J. Duke, A. Jain, V. J. Reddi, J. Jeffries, J. Li, N. Kreeger, I. Naranjo-Alcazar, M. Slaney, T. Warden, et al., “TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems,” in Proc. Conf. Mach. Learn. Syst. (MLSys), 2021, pp. 800–811.

[2] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, C. Gan, and S. Han, “On-Device Training Under 256KB Memory,” in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, 2022, pp. 22941–22954.

[3] L. Ravaglia, M. Rusci, A. Bonetti, and E. Farella, “TinyOL: TinyML with Online-Learning on Microcontrollers,” in Proc. Int. Joint Conf. Neural Netw. (IJCNN), 2022, pp. 1–8.

[4] F. Conti, L. Cavigelli, G. Rutishauser, and L. Benini, “PULP-TrainLib: Enabling On-Device Deep Learning Training on Parallel Ultra-Low-Power RISC-V Multi-Cores,” IEEE Trans. Very Large Scale Integr. (VLSI) Syst., vol. 31, no. 5, pp. 687–700, May 2023.

[5] J. Lin, L. Zhu, W.-M. Chen, W.-C. Wang, and S. Han, “Tiny Machine Learning: Progress and Futures,” IEEE Circuits and Systems Magazine, vol. 24, no. 2, pp. 8–31, Second Quarter 2024.

GL BAJAJ
Group of Institutions
Mathura
FIND YOUR SPARK
Approved by AICTE & Affiliated to AKTU